IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

X62 - Optimisation pour l'aide à la décision
TP n° 2
Descente de gradient

La *descente de gradient* est une heuristique de recherche d'un minimum local d'une fonction **dérivable** à valeurs réelles. Rappelons-en le principe général :

- On part d'un point initial P du domaine de définition de la fonction f à minimiser, point a priori quelconque, mais dont on espère qu'il n'est pas trop loin du minimum recherché.
- On calcule le vecteur gradient de f au point P, qui donne la direction de plus grande croissance (ou *plus grande pente*) de la fonction au point P, si bien que la **direction opposée** est celle de plus grande décroissance, et on espère qu'elle mène à un minimum.
- On déplace donc le point « courant » dans cette direction : l'amplitude (plus précisément : la norme) de ce déplacement dépend à la fois du vecteur gradient et d'un coefficient réel appelé *taux d'apprentissage* (`learning_rate`) qui est un paramètre important de la méthode : `step = -gradient*learning_rate`.
- On répète le calcul du gradient et le déplacement du point courant jusqu'à ce que la norme du déplacement (ou la norme du gradient) soit pratiquement nul : concrètement on s'arrête lorsque le pas est inférieur à un certain seuil (`tolerance`) qui est un paramètre de la méthode.

On verra que le choix du taux d'apprentissage est un point délicat de la mise en œuvre de la méthode. Par soucis de simplicité, nous travaillerons avec un taux constant, mais notez bien qu'on pourrait aussi opter pour un taux variable qui s'adapterait aux autres paramètres de l'algorithme.

Le cœur du TP est la définition d'une fonction (ou plutôt de variantes de cette fonction, déterminées par le contexte particulier)

```
def gradient_descent(diff, start, learning_rate, n_iter=50, tolerance=1e-06):
    '''fonction effectuant la descente de gradient sur la fonction de dérivée diff
    et retournant la liste des points de calcul du gradient'''
```

qui effectue la méthode du gradient avec les paramètres suivants :

- `diff` est la fonction calculant les dérivées de la fonction à minimiser ;
- `start` le point de départ de l'algorithme ;
- `learning_rate` le taux d'apprentissage (valeur par défaut à `0.1`) ;
- `n_iter` le nombre maximal d'itérations (`50` par défaut) ;
- `tolerance` qui permet de stopper l'algorithme si l'amplitude du déplacement lors d'une itération est inférieure à cette valeur (par défaut `1e-06`).

1 Expérimentation de la méthode en dimension 1

Afin de bien saisir le principe de la méthode, nous allons d'abord l'expérimenter dans le cas des fonctions d'une variable réelle, où l'intuition géométrique est particulièrement accessible.

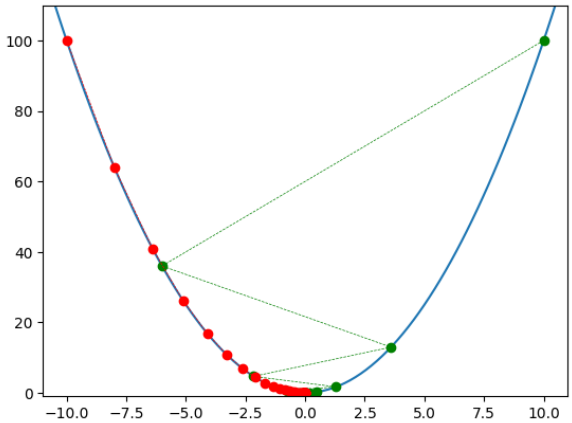
1.1 Parabole

On part de la fonction $f(x) = x^2$ qui a évidemment un unique minimum, local et global, en $x = 0$.

Testez la fonction `gradient_descent` dans ce cas, et observez que la convergence de la méthode ne dépend que du taux d'apprentissage (et non du point de départ).

Pour quelles valeurs de ce taux d'apprentissage a-t-on convergence ? existe-t-il une ou des valeurs optimales ?

Représentez graphiquement l'évolution de l'algorithme de la manière suivante



1.2 Quartique

Répétez le travail précédent avec la fonction (polynôme de degré 4) $g(x) = x^4 + 3x^3 - 4x$.

Observez la grande sensibilité de la convergence à la valeur du `learning_rate`.

Notez aussi que même en cas de convergence, il y a plusieurs limites possibles : lesquelles et pourquoi ?

La limite dépend-elle aussi de point de départ ?

Trouvez de bonnes valeurs des paramètres, c'est-à-dire qui fournissent la convergence vers l'unique minimum global de g.

1.3 Fonction Γ

Dans les exemples précédents la fonction dérivée s'obtenait au moyen de règles algébriques élémentaires à partir de la fonction à minimiser, mais il n'est pas toujours possible ou souhaitable de s'appuyer sur une formule exacte, ou simple pour le calcul de la dérivée.

Par exemple la fonction Γ d'Euler, définie pour tour réel $z > 0$ par l'intégrale généralisée

$$\Gamma(z) = \int_0^{+\infty} t^{z-1} \exp(-t)\, dt$$

(fonction `math.gamma` pour `python`) est dérivable sur son ensemble de définition, mais il n'y a pas de formule aisément accessible pour calculer cette dérivée.

Dans pareil cas, on peut utiliser une formule numérique d'approximation de la valeur de la dérivée, comme la différence finie centrée

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h},$$

où h est une valeur réelle « petite » (l'approximation est d'autant meilleure que h est petit, en principe).

Déterminez au moyen d'une descente de gradient (une valeur approchée de) l'unique minimum de la fonction Γ sur les réels positifs.

2 Descente de gradient pour la droite des moindres carrés

2.1 Principe

Le modèle le plus simple de relation de dépendance quantitative entre deux variables (réelles) X et Y est le modèle linéaire, autrement dit une relation du type

$$Y \simeq b_0 + b_1 X$$

qui est complètement déterminée par les deux coefficients b_0 et b_1.

Étant donné un ensemble de n valeurs observées (x_i, y_i), on souhaite estimer les « meilleures » valeurs possibles de ces coefficients, ou encore le modèle « le plus compatible » avec les observations. Il y a plusieurs manières de donner un sens précis à cette optimalité, et la plus courante repose sur le critère des *moindres carrés* : il s'agit de trouver

les coefficients qui minimisent les carrés des écarts entre la valeur y_i observée et la valeur $b_0 + b_1 x_i$ prévue par le modèle pour l'ensemble des observations.

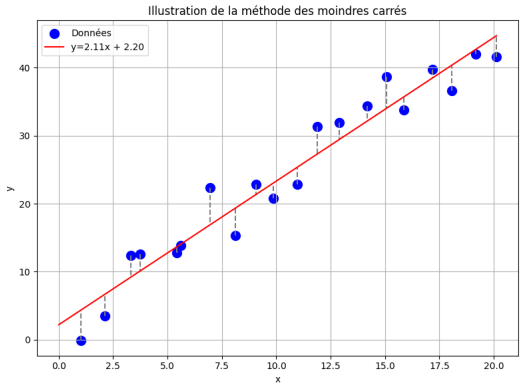
La fonction à minimiser (*fonction de coût*) est donc la somme

$$SSR = \sum_i (y_i - (b_0 + b_1 x_i))^2 \qquad \text{(Sum of Square Residuals)}$$

ou, de manière équivalente, la moyenne des écarts quadratiques

$$MSE = \frac{SSR}{n} \qquad \text{(Mean Squared Error)}$$

La méthode a une interprétation géométrique intuitive comme la recherche de la droite minimisant la somme des carrés des écarts verticaux entre les points observés et la droite :



Pour déterminer le minimum de cette fonction de coût (il est facile de voir qu'il existe un unique minimum local, qui est aussi global), on calcule son gradient, ou encore celui de la fonction $C = MSE/2$:

$$\frac{\partial C}{\partial b_0} = \frac{1}{n} \sum_i (b_0 + b_1 x_i - y_i)$$
$$\frac{\partial C}{\partial b_1} = \frac{1}{n} \sum_i (b_0 + b_1 x_i - y_i) x_i$$

puis on peut résoudre le système linéaire défini par l'annulation de ce gradient, et déterminer ainsi les coefficients b_0 et b_1 qui réalisent le minimum recherché.

De manière alternative, on se propose de procéder par approximations numériques en utilisant la descente de gradient définie plus haut.

Codez les fonctions

```
def ssr_gradient(u, v, b):

def mse(u, v, b):
```

calculant respectivement le gradient de la fonction de coût (explicité ci-dessus) et la valeur du MSE en fonction des données u et v, et du point $b = (b_0, b_1)$.

Codez ensuite la fonction

```
def lin_reg_descent(u, v, learn_rate=0.1, n_iter=50, tolerance=1e-06):
```

effectuant la descente de gradient proprement dite.

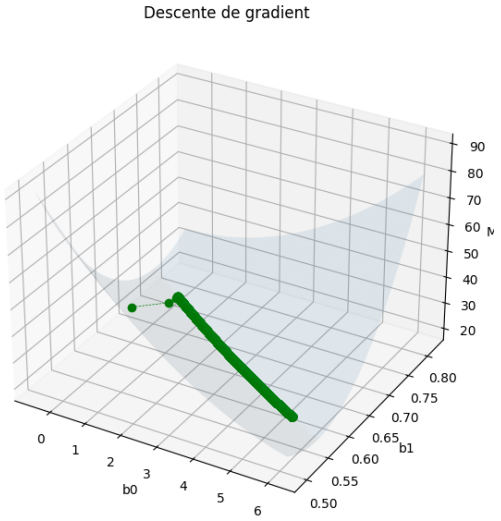
- Les paramètres `u` et `v` contiennent respectivement les données (x_i) et (y_i) (sous forme de `np.array` par exemple).
- Les paramètres `learn_rate`, `n_iter` et `tolerance` ont la même signification que dans les sections précédentes.
- Notez bien que les valeurs par défaut des paramètres ne sont pas nécessairement adaptés à la bonne convergence de l'algorithme en général.

- Il faut aussi décider d'un point initial pour la descente, mais ce n'est pas très important ici, car n'importe quel point de départ convient : pourquoi ?
- La fonction retourne la suite des points parcourus par la descente de gradient.

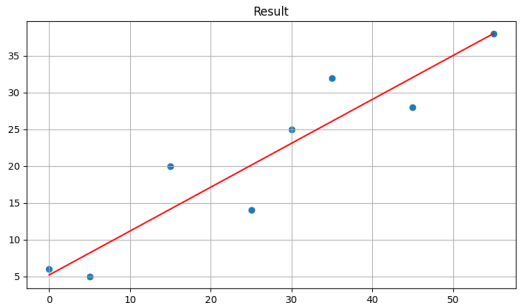
Pour tester la descente on suggère de commencer par utiliser les données suivantes

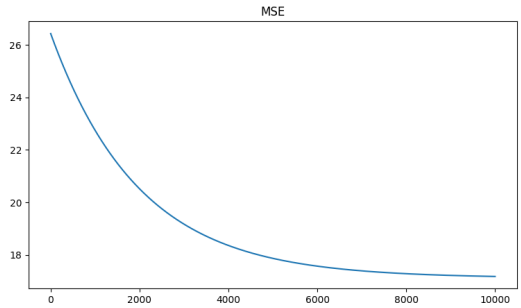
```
u_1 = np.array([-1.2, 1.5 , -0.9, 1.2 ])
v_1 = np.array([-1.1, 1.2, -0.6, 0.9])
```

Afin de contrôler le bon fonctionnement de l'algorithme, on génèrera d'une part un graphique permettant de visualiser la descente sur le graphe de la fonction de coût, et l'évolution de la valeur du coût au fil des itérations :



d'autre part les données et la droite (dite « droite de régression ») obtenue :





Finalement, modifiez les données de test, par exemple avec

```
u_2 = np.array([5, 15, 25, 35, 45, 55, 0, 30])
v_2 = np.array([5, 20, 14, 32, 28, 38, 6, 25])
```

et observez les variations dans la vitesse de convergence.

2.2 Un exemple sur des données réelles

Afin d'illustrer l'utilisation de la descente de gradient dans le cadre de la méthode des moindres carrés sur des données réelles, téléchargez les données se trouvant ici

https ://seafile.unistra.fr/f/f41b4077b8b640539da4/ ?dl=1.

Le jeu de données contient 9568 points de données collectés auprès d'une centrale électrique à cycle combiné sur une période de 6 ans (2006-2011), lorsque la centrale fonctionnait à pleine charge. Les caractéristiques comprennent des variables ambiantes moyennes horaires telles que la température ambiante (AT), la pression ambiante (AP), l'humidité relative (RH) et le vide d'échappement (V), utilisées pour prédire la production nette d'énergie électrique horaire (PE) de la centrale.

On utilise la bibliothèque `pandas` pour importer les données :

```python
import pandas as pd

# Charger les données depuis le fichier local
data = pd.read_excel("Folds5x2_pp.xlsx")

# Afficher les premières lignes du DataFrame
print(data.head())
```

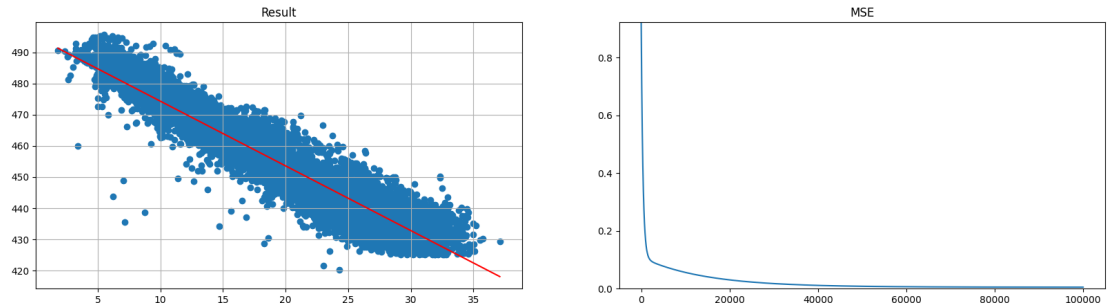
Observez l'hétérogénéité des valeurs et variations des différentes variables. Afin de faciliter la recherche des bons paramètres (initialisation de la recherche, `learning_rate`, etc.), et de garantir la convergence de l'algorithme, il sera probablement nécessaire de normaliser les données (*centrage* et *réduction*, au sens de M31) avant d'effectuer la descente de gradient .

Recherchez le modèle linéaire optimal pour les données PE et AT

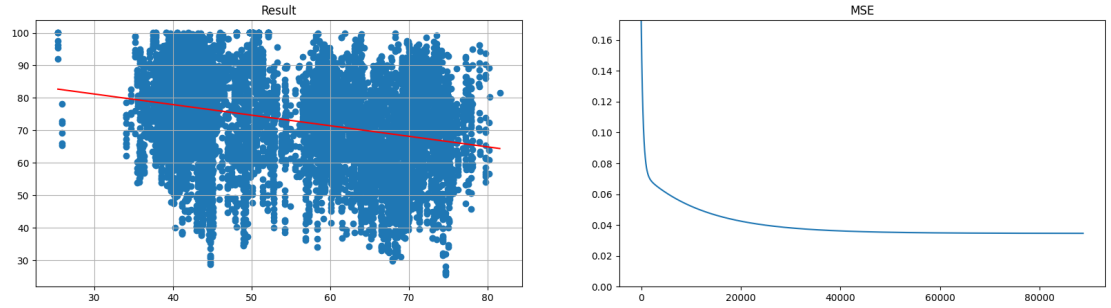
```python
# On choisit les données PE et AT qui sont linéairement corrélées
u = data["PE"].to_numpy()
v = data["AT"].to_numpy()
```

Puis recommencez pour les données V et RH.

Afin de visualiser le résultat, on affichera l'ensemble de données et la droite de régression correspondante, ainsi que l'évolution de la fonction de perte (MSE) au fil des itérations :



Régression avec les valeurs de PE et AT



Régression avec les valeurs de V et RH

On remarque que dans un cas les données ne semblent pas linéairement corrélées et on peut se poser la question de la validité/pertinence d'un modèle linéaire. On rappelle (voir M61) qu'une mesure classique de la validité d'un modèle linéaire est le coefficient de détermination, noté R^2, et défini comme le quotient de la variance prédite expliquée par le modèle, par la variance observée de cette variable.

Codez la fonction

```python
def R2(u, v, b):
```

qui calcule le coefficient de détermination R^2 pour des séries de données `u` et `v` et les coefficients de régression $\mathtt{b} = [\mathtt{b}_0, \mathtt{b}_1]$, puis évaluez ce coefficient pour les deux exemples traités ci-dessus.

2.3 Descente de gradient stochastique

La *descente de gradient stochastique* (DGS) est une adaptation de la descente de gradient, où l'on choisit aléatoirement un sous-ensemble d'observations pour le calcul du gradient de la fonction de coût, et la mise à jour des coefficients.

- Dans la *DGS en ligne*, le gradient est estimé et les variables mises à jour pour chaque choix successif d'observation (point de données).
- Dans la *DGS par lots*, on divise aléatoirement l'ensemble des observations en lots (*batches*) de taille donnée, et on met à jour les variables en utilisant le gradient calculé sur chaque lot.
- Chaque passage à travers l'ensemble des lots (et donc des observations) est appelé une *epoch* (ou *époque* si l'on préfère la traduction française).

L'objectif de l'approche stochastique est d'accélérer la convergence de la descente de gradient

Adaptez la fonction de descente de gradient pour travailler par lots (leur taille `batch_size` devenant un paramètre de la méthode, lui-même susceptible d'optimisation).

Essayez de comparer l'approche directe et l'approche stochastique, en terme de vitesse de convergence.